Esercizio di Crittografia

Il codice **HSNFRGH** è un buon esempio di messaggio cifrato con una tecnica molto semplice: il **cifrario di Cesare**, cioè uno spostamento delle lettere nell'alfabeto. Ecco una breve relazione sui passaggi per decifrarlo.

La stringa è composta solo da lettere maiuscole e non contiene simboli strani o numeri. Questo suggerisce un cifrario classico di sostituzione, molto probabilmente uno **shift (spostamento)**.
La stringa è composta solo da lettere maiuscole e non contiene simboli strani o numeri. Questo suggerisce un cifrario classico di sostituzione, molto probabilmente uno **shift (spostamento)**.

Tentativi di decodifica applicando vari shift all'indietro:

Con 5 posizioni

- H → **C**
- S → **N**
- N → **I**
- F → **A**
- R → **M**
- G → **B**
- H → **C**

Con uno spostamento di 5 indietro, otteniamo: C N I A M B C

Con 3 posizioni

- H → **E**
- S → **P**
- N → **I**
- F → **C**
- R → **O**
- G → **D**
- H → E

Con uno spostamento di 3 indietro, otteniamo: E P I C O D E

La parola è plausibile quindi è stato trovato lo schema applicato.

Nel secondo caso, dobbiamo trovare il testo in chiaro della stringa:

QWJhIHZ6b2VidHl2bmdyIHB1ciB6ciBhciBucHBiZXRi

La stringa contiene lettere maiuscole, minuscole e non ha simboli casuali: questo è tipico di una codifica come la **Base64**.
La decodifica è il processo inverso della codifica: riprende i caratteri ASCII e ricostruisce i bit originali del dato (testo o binario). È un passaggio fondamentale quando ricevi dati "impacchettati" per il trasporto web che devono tornare nel loro formato originale.

1. Ogni carattere della stringa viene cercato nella tabella standard Base64.

Valore	ASCII	Valore	ASCII	Valore	ASCII	Valore	ASCII
0	A	16	Q	32	g	48	w
1	B	17	R	33	h	49	x
2	C	18	S	34	i	50	y
3	D	19	T	35	j	51	z
4	E	20	U	36	k	52	0
5	F	21	V	37	l	53	1
6	G	22	W	38	m	54	2
7	H	23	X	39	n	55	3
8	I	24	Y	40	o	56	4
9	J	25	Z	41	p	57	5
10	K	26	a	42	q	58	6
11	L	27	b	43	r	59	7
12	M	28	c	44	s	60	8
13	N	29	d	45	t	61	9
14	O	30	e	46	u	62	+
15	P	31	f	47	v	63	/

Q —> 16
W —> 22
J —> 9
h —> 33…. e cosi via

2. Trasformare in Binario (6 bit): Ogni indice viene convertito nel suo equivalente binario a 6 bit.

16 —> 010000
22 —> 010110
9 —> 001001
33 —> 100001

3. Raggruppamento (8 bit): I bit ottenuti vengono uniti in una lunga sequenza e poi spezzati in gruppi da 8 bit (1 byte).

Sequenza: 01000001 01100010 0110001……

4. Traduzione in caratteri: Ogni gruppo di 8 bit viene convertito nel carattere corrispondente secondo la codifica UTF-8 o ASCII.

01000001 —> 65 (Decimale) —> A
01100010 —> 98 (Decimale) —> b
01100001 —> 97 (Decimale) —> a

Per farlo velocemente si possono usare risorse online oppure dal terminale Linux basta digitare il comando

echo “tua_stringa” | base64 —decode

Una volta decifrato il risultato sarà:

Aba vzobetyvngr pur zr ar nppbetb

Sembra evidente che il messaggio contiene ancora una cifratura.
Ipotizzando sia stato usato lo stesso metodo del primo esempio (cifrario di Cesare) con diverse prove si scopre essere un ROT13 (ogni lettera si sposta di 13 posizioni).

Risultato finale:

Non imbrogliate che me ne accorgo

Per velocizzare e semplificare il tutto è stato scritto un piccolo programma in Python che esegue le due operazioni.

Welcome | import base64.py

Users > lex > Desktop > import base64.py > ...

```python
import base64
import codecs

# La stringa codificata
stringa_b64 = "QWJhIHZ6b2VidHl2bmdyIHB1ciB6ciBhciBucHBiZXRi"

# 1. Converte la stringa in byte
dati_byte = stringa_b64.encode("ascii")

# 2. Decifra i byte
byte_decodificati = base64.b64decode(dati_byte)

# 3. Trasforma i byte decifrati in una stringa leggibile (UTF-8)
testo_finale = byte_decodificati.decode("ascii")


def rot13(testo):
    return codecs.encode(testo, 'rot_13')

testo_originale = testo_finale
testo_codificato = rot13(testo_originale)

print()
print("Testo originale: " , stringa_b64)
print()
print(f"Codifica Base64: {testo_originale}")
print()
print(f"Risultato finale con ROT13:  {testo_codificato}")
print()
```

PROBLEMS | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS

```
lex@MacBook-Air-di-Alex ~ % /usr/local/bin/python3 "/Users/lex/Desktop/import base64.py"
Testo originale:  QWJhIHZ6b2VidHl2bmdyIHB1ciB6ciBhciBucHBiZXRi

Codifica Base64: Aba vzoebtyvngr pur zr ar nppbetb

Risultato finale con ROT13:  Non imbrogliate che me ne accorgo

lex@MacBook-Air-di-Alex ~ %
```

zsh | Python

0 0 | Ln 19, Col 1 | Spaces: 4 | UTF-8 | LF | { } Python | 3.14.4 | Python 3.14.4 (homebrew)